Student 29 **Perarasan V** 192521404RD 10 / 10 submitted

Q1. Operators in Python

ANSWER

Types of Operators in Python:
1. Arithmetic Operators – Used for mathematical operations.

* + Addition: 5 + 3 = 8
* - Subtraction: 5 - 3 = 2
* * Multiplication: 5 * 3 = 15
* / Division: 5 / 2 = 2.5
* % Modulus: 5 % 2 = 1
* ** Exponentiation: 2 ** 3 = 8
* // Floor Division: 5 // 2 = 2

2. Comparison (Relational) Operators – Compare two values and return True or False.

* == Equal to: 5 == 5 → True
* != Not equal to: 5 != 3 → True
* >, <, >=, <=

3. Logical Operators – Used to combine conditional statements.

* and: True and False → False
* or: True or False → True
* not: not True → False

4. Assignment Operators – Used to assign values to variables.

* = : x = 5
* += : x += 3 (x = x + 3)
* -=, *=, /=

* = : x = 5
* += : x += 3 (x = x + 3)
* -=, *=, /=

5. Bitwise Operators – Operate on binary numbers.

* & AND, | OR, ^ XOR, ~ NOT, << Left shift, >> Right shift

6. Membership Operators – Test if a value is in a sequence.

* in: 3 in [1, 2, 3] → True
* not in: 5 not in [1, 2, 3] → True

7. Identity Operators – Check if two variables point to the same object.

* is: x is y
* is not: x is not y

Q2. Operators

ANSWER

In Python, the operators /, //, %, and ** are arithmetic operators, each serving a different purpose:

1. / (Division): Performs normal division and always returns a float result.
Example: 10 / 3 = 3.3333...

2. // (Floor Division): Divides and returns only the integer part (rounds down to the nearest whole number), discarding the decimal.
Example: 10 // 3 = 3

3. % (Modulus): Returns the remainder after division.
Example: 10 % 3 = 1 (because 10 divided by 3 gives remainder 1)

4. (Exponentiation): Raises a number to the power of another number.
Example: 2 3 = 8 (which means 2 × 2 × 2)

Quick Summary Table:

* 10 / 3 → 3.3333 (float division)
* 10 // 3 → 3 (floor division)
* 10 % 3 → 1 (remainder)
* 2 ** 3 → 8 (power)

Q3. for and while loop

ANSWER

while condition:
    statement(s)A for loop in Python is used to iterate over a sequence (such as a list, tuple, string, or range) for a fixed number of times. It automatically moves through each element one by one until the sequence is exhausted.

Syntax of for loop:

for variable in sequence:

Example:

```
for i in range(5):
    print(i)
```

This prints 0 to 4.

A while loop, on the other hand, repeats a block of code as long as a given condition remains True. It does not iterate over a sequence directly — it checks a condition before each iteration.

Syntax of while loop:

```
while condition:
    statement(s)
```

Example:

```
i = 0
while i < 5:
    print(i)
    i += 1
```

Key Differences:

1. A for loop is used when the number of iterations is known in advance, while a while loop is used when the number of iterations is unknown and depends on a condition.
2. A for loop automatically iterates over a sequence, whereas a while loop requires manual updating of the condition variable (e.g., incrementing i).
3. A while loop can lead to an infinite loop if the condition never becomes False, which is less common with a for loop.

Q4. range()

ANSWER

Differences between range(5), range(1, 6), and range(1, 10, 2):

1. range(5) – Takes only one argument (stop). It generates numbers from 0 up to (but not including) 5.

Output: 0, 1, 2, 3, 4

2. range(1, 6) – Takes two arguments (start, stop). It generates numbers from 1 up to (but not including) 6.

Output: 1, 2, 3, 4, 5

3. range(1, 10, 2) – Takes three arguments (start, stop, step). It generates numbers from 1 up to (but not including) 10, skipping every 2 numbers.

Output: 1, 3, 5, 7, 9

Example:

```
for i in range(1, 10, 2):
    print(i)
```

This will print: 1, 3, 5, 7, 9

In summary, range() makes looping easier and more efficient by generating number sequences with a defined start, stop, and step value.

Q5. String

ANSWER

A string in Python is a sequence of characters enclosed within single quotes (' '), double quotes (" "), or triple quotes (''' ''' or """ """). It is used to store and manipulate textual data. For example: name = "Python" is a string.

A string in Python is IMMUTABLE. This means that once a string is created, its contents cannot be changed or modified.

Justification:

1. If we try to change a character in a string directly, Python will raise a TypeError. For example:

```
s = "hello"
s[0] = "H"  # This gives TypeError: 'str' object does not support item assignment
```

2. When we perform operations like concatenation or slicing on a string, Python does not modify the original string. Instead, it creates a new string object in memory.
For example:
s = "hello"
s = s + " world" # A new string is created, the original "hello" is not changed

3. Since strings are immutable, they can be used as keys in dictionaries and elements in sets, which require hashable (unchangeable) objects.

Therefore, strings are immutable objects in Python.

Q6. String Slice

ANSWER

Given: s = "Python"

1. s[1:5] → "ytho"
(Characters from index 1 up to but not including index 5)

2. s[::-1] → "nohtyP"
(Reverses the entire string using a step of -1)

3. s[::2] → "Pto"
(Every 2nd character starting from index 0: P, t, o)

4. s[-3:] → "hon"
(Last 3 characters of the string, starting from index -3)

Q7. List

ANSWER

A list in Python is a built-in data structure that stores an ordered collection of items. The items can be of different types (e.g., integers, strings, even other lists). Lists are created using square brackets, like: my_list = [1, "a", 3.5].

A string, on the other hand, is also an ordered collection, but it stores only characters (text). Strings are created using quotes, like: my_string = "hello".

Key differences:

* Data type: List holds any type of elements; String holds only characters.
* Mutability: Lists are mutable (you can change/add/remove items). Strings are immutable (you cannot change a character in-place).
* Syntax: Lists use [ ], strings use ' ' or " ".
* Operations: Lists support methods like append(), remove(), etc.; strings support methods like upper(), split(), but not append().

Q8. List Slice

ANSWER

```
print(L[2:6])   # [30, 40, 50, 60]
print(L[:4])    # [10, 20, 30, 40]
print(L[::2])   # [10, 30, 50, 70]
print(L[::-1])  # [70, 60, 50, 40, 30, 20, 10]
print(L[-4:-1]) # [40, 50, 60]
```

Q9. Index

ANSWER

In Python, indexing is how we access elements in a sequence (like a string or list).

* Positive indexing starts from the beginning: the first element has index 0, second has 1, and so on.
* Negative indexing starts from the end: the last element has index -1, second last has -2, and so on.

Example using a string:

```
s = "Python"
print(s[0])   # 'P'  (positive indexing)
print(s[1])   # 'y'
print(s[-1])  # 'n'  (negative indexing)
```

```
print(s[-1])  # 'n'  (negative indexing)
print(s[-2])  # 'o'
```

Q10. length

ANSWER

In Python, len() is a built-in function that returns the number of items in a collection (e.g., length of a string, list, tuple). count() is a method that returns how many times a specific element appears in a sequence (e.g., list.count(x) counts occurrences of x). So len() gives total size, while count() gives frequency of a particular value.